

William Jackson

Senior Software Engineer – Python(Django/Fast API) AWS

Wellington, Nevada, United States | +1 (786) 949-7561 | williamjackson.pro@outlook.com | [linkedin.com/in/william-jackson-75b9343b6](https://www.linkedin.com/in/william-jackson-75b9343b6)

PROFESSIONAL SUMMARY

Senior Software Engineer with ~10 years delivering **Python** backend platforms for **network operations**, **data products**, and **reliability-focused SaaS**, building secure and observable systems with **Python 2.7–3.11**, **FastAPI 0.9x–0.10x**, **Django 1.11–4.x**, and modern cloud services. Strong track record translating ambiguous business requests into stable **REST** and **GraphQL** contracts, replay-safe workflows, and schema-governed pipelines that protect uptime, latency, and auditability. Hands-on across architecture, implementation, production support, and postmortem-driven improvement, with consistent success reducing alert noise, tightening error budgets, and modernizing legacy services without breaking downstream consumers.

SKILLS

- **Backend & APIs** — **Python 2.7–3.11**, **FastAPI 0.9x–0.10x**, **Django 1.11–4.x**, SQL, REST APIs, GraphQL, microservices, event-driven architecture, background jobs, WebSockets, rate limiting, idempotency, caching strategies, Bash
- **Data, Messaging & Search** — **PostgreSQL**, **MySQL**, **SQL Server**, **Redis 5.x–6.x**, **MongoDB**, **DynamoDB**, **AWS SQS/SNS**, **Kafka**, **RabbitMQ**, **Elasticsearch 5.x–8.x**, **OpenSearch**, **Redshift**, **BigQuery**
- **Cloud & DevOps** — **AWS (EKS 1.2x–1.27, ECS, EC2, Lambda, API Gateway, S3, CloudFront, RDS, ElastiCache, EventBridge, Step Functions, IAM)**, **Azure (AKS, App Service, Azure SQL, Service Bus, Key Vault, Application Insights)**, **GCP (GKE, Cloud Run, Pub/Sub, Cloud Storage, BigQuery)**, **Docker**, **Kubernetes**, **Helm 3.x**, **Argo CD 2.x**, **GitOps**, **Terraform 1.x**, **CloudFormation**, **GitHub Actions**, **GitLab CI**, **Jenkins**, **Azure DevOps**, **CircleCI**
- **Observability, Quality & Security** — **OpenTelemetry 1.x**, **Datadog**, **Prometheus 2.x**, **Grafana 7.x–10.x**, **ELK/EFK**, **CloudWatch**, structured logging, API contract testing, **OAuth2**, **OpenID Connect**, **JWT**, **Okta**, **Auth0**, **AWS Cognito**, **RBAC**, **ABAC**, audit logging

WORK HISTORY

Senior Software Engineer

February 2021 to December 2025

Viaro Networks Inc. - Delaware, United States

- Led modernization of a critical **Python 3.11** service portfolio using **FastAPI 0.103**, where explicit request schemas, bounded payload handling, and stable error contracts reduced client integration defects by more than **35%** across internal and partner-facing APIs.
- Resolved a costly double-billing production issue caused by duplicate event delivery by implementing **idempotency keys**, **SQS FIFO** ordering controls, conditional persistence in **DynamoDB**, and replay validation in CI, cutting financial reconciliation incidents by over **90%**.
- Eliminated a hard-to-reproduce latency spike in high-volume endpoints by replacing N+1 access patterns with batched retrieval on **PostgreSQL 13**, adding covering indexes, and validating execution plans before release, improving p95 response times by roughly **42%**.
- Migrated legacy background processing from **Celery 4.4** to **Celery 5.3**, introducing safer task routing, structured retry policies, and visibility timeout discipline that reduced poison-message loops while preserving operational ordering guarantees.
- Implemented a new **deterministic replay** capability with **AWS Step Functions** and durable checkpointing so operators could safely rerun failed workflows without duplicating side effects, significantly reducing manual remediation effort during incident recovery.
- Reduced alert fatigue by standardizing telemetry with **OpenTelemetry 1.x**, correlated traces, and SLO-based alerting in **Datadog**, improving triage speed by more than **50%** and making noisy infrastructure symptoms easier to separate from real customer impact.
- Fixed a subtle data corruption defect caused by timezone drift during ingestion by enforcing UTC normalization, standardizing **ISO-8601** serialization, and centralizing time conversion logic behind a single validated library boundary.

- Strengthened tenant isolation and privileged access controls through **OAuth2**, **OpenID Connect**, **JWT**, **RBAC**, and **ABAC**, then added immutable audit trails for sensitive actions to improve compliance readiness and investigation accuracy.
- Migrated containerized delivery from manually maintained AMI-based deployments to **EKS 1.27** with **Helm 3.12** and **Argo CD 2.8**, eliminating configuration drift and enabling safe rollback workflows for multi-service releases.
- Improved throughput and infrastructure efficiency by moving hot read paths behind **Redis 6.2** with explicit TTL rules, targeted invalidation events, and contract-level validation, reducing repeated database pressure by more than **30%** without sacrificing correctness.
- Standardized infrastructure delivery with **Terraform 1.6**, adding plan review gates, environment promotion controls, and safer rollback practices so production changes became more predictable during high-pressure release windows.
- Built contract-first integration coverage using **Pact-style** validation between backend services and consuming clients, which allowed downstream teams to release independently and sharply reduced surprise production breakages after interface changes.

Software Engineer

October 2018 to January 2021

Avantia, Inc. - Ohio, United States

- Built reproducible data pipeline foundations in **Python 3.8** using **Airflow 1.10** DAG patterns, versioned configurations, and checksum-based controls so reruns and backfills remained deterministic even under changing upstream inputs.
- Fixed a silent accuracy defect in aggregation workflows by replacing floating-point math with **Decimal** handling, enforcing scale discipline, and adding regression coverage for edge conditions that had previously escaped review.
- Refactored a legacy ETL monolith into smaller, composable jobs with explicit inputs and outputs, then introduced selective reprocessing controls so partial failures no longer forced full-batch reruns and delayed delivery windows.
- Implemented queue-driven execution with **AWS SQS** and worker pools, adding exponential backoff, dead-letter handling, and deduplication safeguards that stabilized burst workloads and reduced repeated failure cascades during partner import spikes.
- Improved high-read API performance by introducing **Redis 5.x** caching with explicit invalidation events instead of guess-based expiry, which protected freshness while reducing repetitive database scans by approximately **28%**.
- Reduced export failures by validating schemas at service boundaries, emitting typed error payloads, and enforcing fail-fast checks so malformed partner data surfaced within minutes instead of remaining hidden until downstream consumers failed.
- Implemented new analytical export workflows into **BigQuery**, using consistent partition keys and disciplined load semantics to deliver stable tables that analysts could trust without frequent manual repair cycles.
- Strengthened secrets handling through IAM-scoped access, vault-oriented workflows, and safer local development defaults, reducing accidental key exposure risk while preserving developer productivity in shared environments.
- Stabilized reporting visibility by instrumenting services with **Prometheus 2.26** metrics and **Grafana 7.5** alerting, shifting the team from reactive debugging toward proactive failure detection and trend-based operational review.
- Standardized CI behavior with **GitHub Actions**, Dockerized services, and reproducible test fixtures so local environment differences stopped blocking releases and incident-response patch deployment.
- Introduced end-to-end integration coverage for transforms and delivery contracts, preventing schema drift from breaking downstream consumers in **Redshift** and partner-managed reporting workflows.
- Partnered closely with non-technical stakeholders through demos, acceptance checkpoints, and scoped release plans, turning vague requirements into incremental deliveries that reduced scope blowups and improved delivery confidence.

Software Developer
ArnAmy, Inc. - Florida, United States

March 2016 to September 2018

- Built ETL workflows in **Python 3.6** that normalized event streams into stable identifiers and consistent timestamp semantics, creating a more trustworthy analytics foundation and reducing manual cleanup effort across downstream reporting.
- Fixed a critical out-of-order event bug where stale records overwrote newer data by introducing monotonic version checks, reconciliation reports, and safer upsert rules that materially improved dataset reliability.
- Implemented internal service endpoints in **Django 2.0** with pagination, filtering, and defensive usage limits so consumers could self-serve operational datasets without overwhelming shared database resources.
- Improved hot-path query performance across **PostgreSQL 9.6** and **MySQL 5.7** by rewriting expensive joins, introducing targeted indexes, and validating gains with production-like benchmark traffic before release.
- Shipped a new search export capability for **Elasticsearch 5.6**, producing index-friendly documents and disciplined mappings that kept query behavior stable as data volume and search complexity increased.
- Reduced mean time to diagnosis by adding structured logs and correlation identifiers to pipeline and API paths, giving operators the context needed to isolate failures without attaching debuggers in live environments.
- Implemented safer retry, timeout, and fallback policies for unstable upstream integrations, preventing flaky third-party endpoints from causing prolonged incidents and blocked processing windows.
- Introduced **Docker**-based development workflows to reduce environment drift, accelerate onboarding, and ensure CI executed against the same dependency expectations developers used locally.
- Added **Redis**-backed caching for read-heavy paths with explicit key strategy and freshness rules, improving endpoint responsiveness while preserving predictable invalidation behavior.
- Turned vague product requests into staged deliverables with explicit acceptance criteria, which improved release predictability and lowered rework during feature-heavy delivery cycles.
- Built reconciliation jobs comparing source-of-truth feeds to derived tables, converting data quality issues into actionable reports and preventing silent downstream surprises from reaching business users.

Junior Software Developer
Centric Tech Inc. - New Jersey, United States

October 2014 to February 2016

- Delivered robust data-cleaning and ingestion utilities in **Python 2.7**, using defensive parsing and repeatable validation outputs to replace manual spreadsheet-driven triage with auditable processing workflows.
- Fixed a recurring ingestion failure caused by inconsistent source encodings by normalizing Unicode handling, defining input contracts, and preserving sanitized originals for auditability and later investigation.
- Added safer network handling for early partner integrations through bounded retries, explicit timeouts, and clearer error reporting, which reduced transient processing failures during peak activity periods.
- Improved release confidence by comparing outputs against known-good baselines and building lightweight regression checks that caught breaking behavior before changes reached end users.
- Contributed CI hygiene improvements with scripted checks and more consistent packaging practices, reducing “passed locally” surprises and making rollback and review workflows easier for senior engineers.
- Supported production troubleshooting through structured log review and query inspection, learning practical bottleneck isolation methods and documenting root-cause fixes to prevent repeat incidents.
- Documented expected input and output behavior for utilities and scheduled jobs so future changes remained reviewable, maintainable, and easier for new team members to understand quickly.
- Collaborated with senior engineers to break ambiguous assignments into concrete technical steps, improving ownership clarity and making deliverables easier to estimate, review, and validate.

EDUCATION

Bachelor's degree in Computer Science
Stanford University Department of Computer Science

2010 to 2014

- Built strong foundations in **algorithms**, **databases**, **software engineering**, and applied systems design, creating a solid base for later work in distributed services, data modeling, and production reliability.